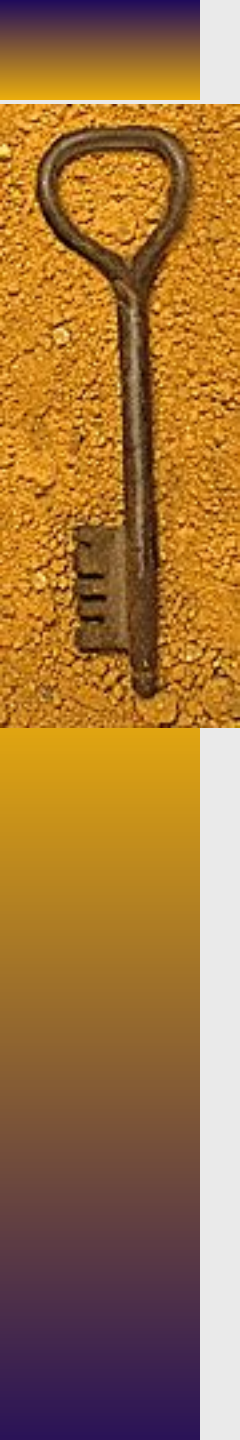




Orientação a Objetos em C++

Prof. Dr. Valter Vieira de Camargo

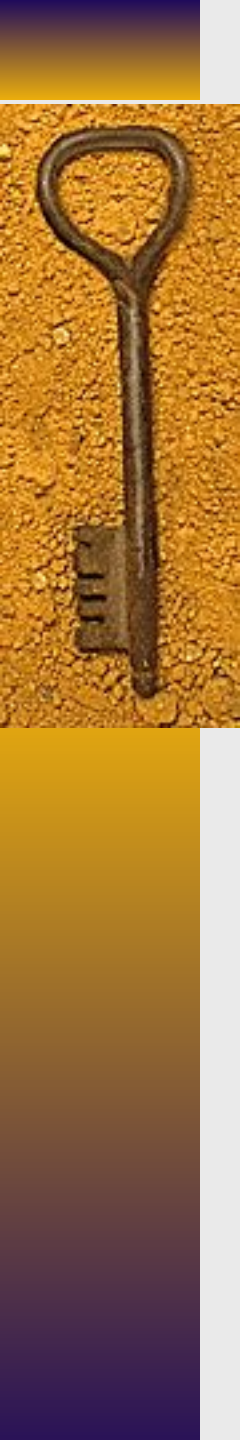
POO - 2026/1



Classes

- ◆ Classes são “tipos”
- ◆ A partir delas podemos criar n instâncias
- ◆ Essas instâncias são chamadas de “objetos”
- ◆ Exemplo
 - Suponha a existência de uma classe chamada Cliente
 - Podemos então criar os objetos/instâncias: Valter, Arthur, João, Pedro, Roberta, etc...





Classes

- ♦ Classes em orientação a objetos são agrupadores de dados e comportamento em uma única caixinha
- ♦ Dados → atributos
- ♦ Comportamento → métodos (funções)
- ♦ Em geral
 - dados devem ser privados à própria classe (você controla o acesso a eles)
 - métodos são públicos, mas podem também serem privados/protegidos

Conceito para o próximo slide

Assinatura e Corpo de Métodos

- ◆ A assinatura de um método é composta por:
 - seu nome
 - seus parâmetros
 - seu tipo de retorno
- ◆ O corpo do método é o seu interior

assinatura
(também
conhecido por
interface)

```
int somar(int a, int b) {  
    return a + b;  
}
```

Corpo

Conceito: Assinatura



...

```
int main() {
```

```
    Calculadora calc; // cria um objeto da classe
```

```
    int resultado = calc.somar(10, 5); // chama o método sem  
    saber como ele é implementado internamente
```

```
    cout << "Resultado: " << resultado << endl;
```

```
    ...
```

```
class Calculadora {
```

```
public:
```

```
    int somar(int a, int b) {  
        return a + b;
```

```
    }
```

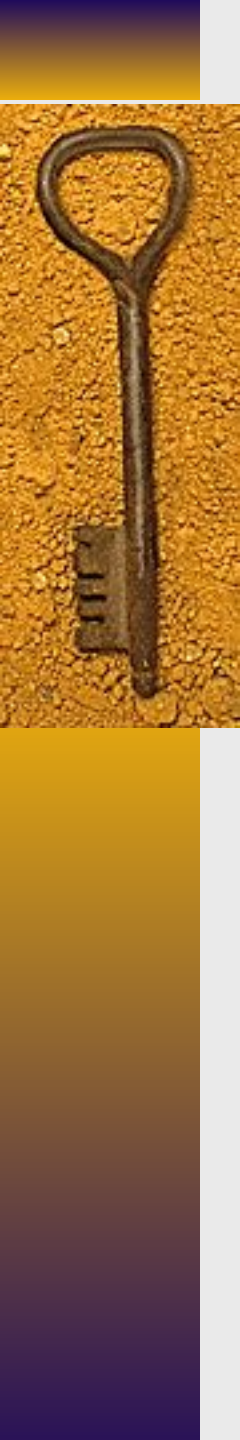
```
};
```

Note algo importante :

Uma coisa é “usar” um método e a outra coisa é “como ele é internamente implementado”.

Quem usa não sabe, e não deve mesmo saber, como ele é implementado.

Como o arquivo de cima (main) usa a classe Calculadora, ele é também chamado de Código Cliente



Classes

- ♦ Toda classe em C++ **normalmente** deve ser declarada em dois arquivos:
 - Um arquivo .h
 - Um arquivo .cpp
- ♦ O arquivo .h (Header)
 - serve como um “contrato” do que a classe é
 - declara o que a classe faz, mas não como ela faz
 - contém apenas **assinaturas** de métodos (sem corpo)
 - contém também os atributos
- ♦ O arquivo .cpp
 - é a implementação de fato da classe
 - é aqui onde o corpo dos métodos são implementados
 - os métodos devem ter corpo (no .h não)
 - pode ter atributos também
- ♦ Altamente recomendado que os dois arquivos tenham o mesmo nome, ex: NotaFiscal.h e NotaFiscal.cpp

Classes

- ♦ Por que declarar a classe em dois arquivos?
 - é uma boa prática separar o contrato (interface) da implementação
 - o “código cliente” (main) usa a classe por meio de sua interface (.h). Assim, pode-se modificar a implementação em um arquivo sem afetar o outro
 - o código cliente só precisa conhecer o arquivo .h
 - Pode fazer tudo no mesmo arquivo ? Até pode, mas a separação entre “interface” e “implementação” sempre é bem vinda





```
class Funcionario {  
private:  
    int codigo;  
    std::string nome;  
    int idade;
```

```
public:  
    Funcionario(int c, std::string n, int i); // Construtor  
    void apresentar(); // Método para exibir informações  
};
```

Funcionario.h

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp



```
class Funcionario {
```

```
private:
```

```
    int codigo;  
    std::string nome;  
    int idade;
```

Modificadores de visibilidade

```
public:
```

```
    Funcionario(int c, std::string n, int i); // Construtor  
    void apresentar(); // Método para exibir informações  
};
```

Funcionario.h

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp



```
class Funcionario {  
private:
```

```
    int codigo;  
    std::string nome;  
    int idade;
```

```
public:
```

```
    Funcionario (int c, std::string n, int i);    // Construtor  
    void apresentar();    // Método para exibir informações  
};
```

Funcionario.h

Atributos e
Assinatura dos Métodos (sem corpo)

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp



```
class Funcionario {  
private:  
    int codigo;  
    std::string nome;  
    int idade;
```

Funcionario.h

```
public:  
    Funcionario (int c, std::string n, int i);    // Construtor  
    void apresentar();    // Método para exibir informações  
};
```

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n,  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Inclusão do .h, que contém a definição da classe.

Funcionario.cpp



```
class Funcionario {  
private:  
    int codigo;  
    std::string nome;  
    int idade;
```

Funcionario.h

```
public:  
    Funcionario (int c, std::string n, int i);    // Construtor  
    void apresentar();    // Método para exibir informações  
};
```

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Concretização dos Métodos
Implementação dos Métodos

Funcionario.cpp



```
class Funcionario {  
private:  
    int codigo;  
    std::string nome;  
    int idade;
```

```
public:  
    Funcionario (int c, std::string n, int i) :  
        codigo(c), nome(n), idade(i) {}  
    void apresentar();  
};
```

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.h

::

O :: chama-se Operador de Resolução de Escopo. Serve para dizer que aquele elemento é o mesmo que foi declarado na classe Funcionario.

Se não colocar, o compilador entenderá que você está declarando um outro método apresentar() de escopo geral/público.

Funcionario.cpp



```
class Funcionario {  
private:  
    int codigo;  
    std::string nome;  
    int idade;
```

Funcionario.h

```
public:  
    Funcionario (int c, std::string n, int i) :  
        codigo(c), nome(n), idade(i) {}  
    void apresentar(); // Método  
};
```

Dentro do arquivo .cpp tem que
incluir o .h

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp



```
class Funcionario {
private:
    int codigo;
    std::string nome;
    int idade;

public:
    Funcionario (int c, std::string n, int i) :
        codigo(c), nome(n), idade(i) {}
    void apresentar(); // Método
};
```

Funcionario.h

Essa inclusão é um copiar e colar do arquivo .h dentro do arquivo .cpp.
Isso ocorre no pré-processamento, uma etapa antes de realmente compilar

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",
    Idade: " << idade << std::endl;
}
```

Funcionario.cpp

O que é um Construtor ?

```
public:  
    Funcionario (int c, std::string n, int i);    // Construtor  
    void apresentar();    // Método para exibir informações  
};
```

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp

- Construtor é um método especial. O objetivo dele é permitir que os atributos de um objeto da classe sejam inicializados com valores.
- Obrigatoriamente tem que ter o mesmo nome da Classe
- Você até pode não declarar nenhum construtor, mas o compilador irá criar um “construtor default” invisível... pq sem ele não tem como criar objetos



```
public:
```

```
Funcionario (int c, std::string n, int i); // Construtor  
void apresentar(); // Método para exibir informações  
};
```

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
: codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp

- Entretanto, o construtor criado pelo compilador é sem parâmetros, não dá para inicializar os atributos do objeto sendo criado com valores específicos

public:

Funcionario (int c, std::string n, int i); // Construtor
void apresentar(); // Método para exibir informações

};

#include "Funcionario.h"

Funcionario::Funcionario(int c, std::string n, int i)
: codigo(c), nome(n), idade(i) {}

void Funcionario::apresentar() {
 std::cout << "Código: " << codigo << ", Nome: " << nome << ",
 Idade: " << idade << std::endl;
}

Funcionario.cpp



```
class Funcionario {
private:
    int codigo;
    std::string nome;
    int idade;

public:
    Funcionario (int c, std::string n, int i) :
        codigo(c), nome(n), idade(i) {} // Método de inicialização
    void apresentar(); // Método de apresentação
};
```

Funcionario.h

Este uso aqui de “:” chama-se *lista de inicialização*. É uma forma de inicialização de construtores mais **eficiente** do que o normal.

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",
    Idade: " << idade << std::endl;
}
```

Funcionario.cpp

Por causa do construtor é possível criar objetos da classe:

```
int main() {  
    Funcionario valter (1, "Carlos", 30);
```

public:

```
    Funcionario (int c, std::string n, int i);    // Construtor  
    void apresentar();    // Método para exibir informações
```

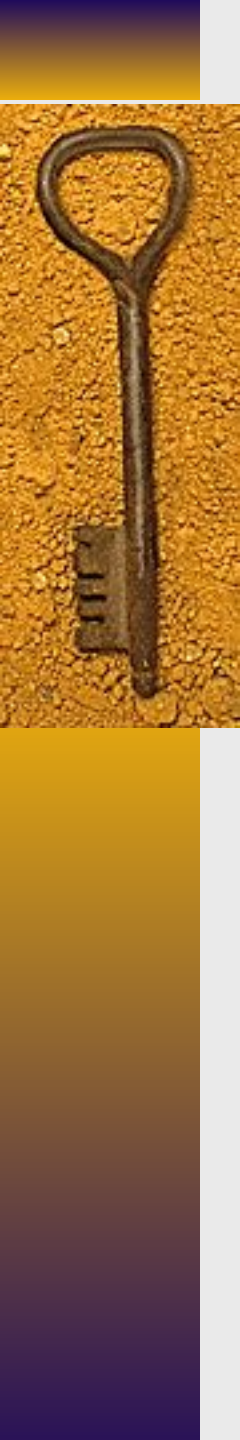
```
};
```

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "Código: " << codigo << ", Nome: " << nome << ",  
    Idade: " << idade << std::endl;  
}
```

Funcionario.cpp



Construtor Sobrecarregado

- ◆ Pode haver mais de um construtor para uma mesma classe
- ◆ O que difere os construtores é o número e tipo dos parâmetros
- ◆ Suponha que deva haver um construtor na qual o usuário informe apenas o nome e a idade do Funcionario e o código seja automaticamente inicializado com zero
- ◆ Isso é útil quando há valores default para determinados atributos
- ◆ Também é útil haver um construtor vazio

Detalhe sobre diretivas...



```
#ifndef FUNCIONARIO_H
#define FUNCIONARIO_H
```

```
#include <iostream>
#include <string>
```

```
class Funcionario {
private:
    int codigo;
    std::string nome;
    int idade;
```

```
public:
    Funcionario(int c, std::string n, int i); // Construtor
    Funcionario();
    void apresentar(); // Método para exibir informações
    void promover(std::string novoCargo);
    void demitir();
    int calcularSalario();
```



```
#endif
```

- # é uma diretiva de pré-processamento (antes de compilar)
- Não fazem parte da linguagem, mas são instruções para o pré-processador
- #ifndef → “se não estiver definida ainda ..”
- Essa diretiva impede que esse arquivo .h seja incluído mais de uma vez (em outros .cpp), o que daria erros de redefinição
- Imagina se um método com o mesmo nome fosse implementado com corpos diferentes em arquivos diferentes
- Deve-se colocar isso em toda definição de classe

Detalhe sobre diretivas...



```
#ifndef FUNCIONARIO_H
#define FUNCIONARIO_H
```

```
#include <iostream>
#include <string>
```

```
class Funcionario {
private:
    int codigo;
    std::string nome;
    int idade;
```

```
public:
    Funcionario(int c, std::string n, int i); // Construtor
    Funcionario();
    void apresentar(); // Método para exibir informações
    void promover(std::string novoCargo);
    void demitir();
    int calcularSalario();
```



```
#endif
```

- ifndef e define sempre trabalham juntos
- o “define” define um booleano para informar se - FUNCIONARIO_h já foi definido (true ou false)
- o ifndef é uma checagem do booleano, se o define for false, então, ele deixa incluir



Como criar objetos
(instâncias) das Classes



Objetos/Instâncias

- Classes estão para tipos, assim como objetos estão para conteúdos ou valores de uma variável
- Exemplo
 - Classe: Funcionário (codigo, nome, cpf)
 - Objetos:

nome do objeto/variável (sempre crie nomes significativos)	valores dos atributos codigo, nome e cpf
f1	1, "valter", 567.999.888-22
ro	2, "Roberta", 888.111.777-22
arthur	3, "Arthur Camargo", 444.555.666.00



Objetos/Instâncias

Há duas formas de criar objetos:

- Forma estática (automática)
- Forma dinâmica (ponteiro)

- Forma estática para a classe Funcionario

```
// arquivo main.cpp
```

```
int main() {
```

```
    Funcionario f (1, "Carlos", 30);
```

```
}
```

- Esta sendo instanciado (criado) um objeto chamado “f” e inicializado com o código 1, com o nome Carlos e com idade 30.
- Este objeto f, pode então ser usado ao longo do código...
- com o “f” pode-se agora chamar os métodos disponíveis na classe Funcionario



Objetos/Instâncias

Há duas formas de criar objetos:

- Forma estática (automática)
- Forma dinâmica (ponteiro)

- Para a classe Funcionario mostrada antes:

```
// arquivo main.cpp
```

```
int main() {
```

```
    Funcionario f (1, "Carlos", 30);
```

```
    f.apresentar()
```

Aqui, o método apresentar() está sendo chamado para exibir os dados do funcionário f.

Objetos/Instâncias

Forma dinâmica

- Uma outra forma de criar objetos é por meio de ponteiros

```
// arquivo main.cpp
```

```
int main() {
```

```
    Funcionario* f = new Funcionario(1, "Carlos", 30);
```

```
    f->apresentar()
```

```
    ...
```

```
    ...
```

```
    delete f;
```

```
}
```

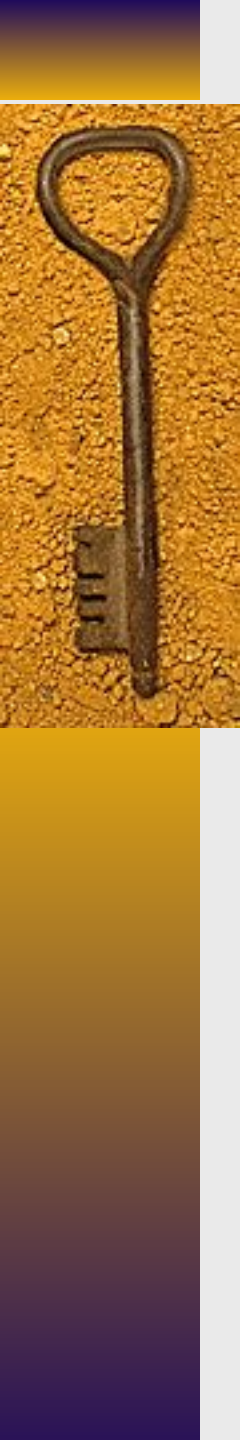
Veja que muda a forma de acessar os membros (atributos e métodos)

Você mesmo precisa liberar a memória



Comparação das duas formas

- Diferença entre as formas de instanciar objetos
- **Forma estática (automática)**
 - o objeto é criado na stack (memória automática)
 - o tempo de vida do objeto está ligado ao bloco de código onde ele foi criado (main ou algum método)
 - a memória é liberada automaticamente quando sai do escopo (não precisa invocar o `delete()` para liberar memória)
 - mais simples e mais seguro
- **Forma dinâmica**
 - o objeto é criado na heap (memória dinâmica)
 - são dois passos, primeiro criar um ponteiro para um tipo e depois alocar memória com o *new*
 - tem que chamar o *delete* para liberar a memória
 - útil quando:
 - não sabe quantos objetos irá criar
 - precisa que o objeto viva além do escopo atual
 - está lidando com listas, pilhas e filas ... (estruturas dinâmicas)



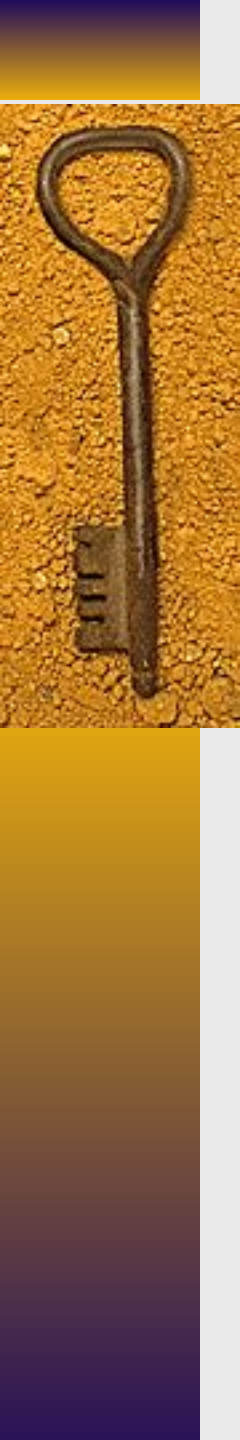
Vamos dar uma melhoria no método apresentar().
Este é o arquivo Funcionario.cpp

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)  
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {  
    std::cout << "--- PRINTANDO DADOS ---" << std::endl;  
    std::cout << "Codigo: " << codigo << std::endl;  
    std::cout << "Nome: " << nome << std::endl;  
    std::cout << "Idade: " << idade << std::endl;  
}
```

Ao chamar o método apresentar(), os dados são impressos na saída



```
main.cpp  Funcionario.h  Funcionario.cpp    
1  #include "Funcionario.h"   
2   
3  int main() {   
4      Funcionario f1 (1, "Carlos", 30);   
5      f1.apresentar();   
6   
7      std::cin.get();   
8      return 0;   
9   
10 }   
11   
12   
13   
--- PRINTANDO DADOS ---   
Codigo: 1   
Nome: Carlos   
Idade: 30
```

Métodos sets() e gets()

- ◆ Toda classe **pode** ter métodos públicos que permitem acessar seus atributos individualmente (se você quiser)
- ◆ Devem ser criados métodos set*() que são utilizados para **alterar** o valor de algum atributo. Ex. setCodigo(int c)
- ◆ Devem ser criados métodos get*() que são utilizados para a obtenção (ler) do valor de algum atributo. Ex. getCodigo()
- ◆ São também conhecidos como métodos “acessores” ou “métodos que permitem o acesso aos atributos”
- ◆ Normalmente são públicos, mas você pode julgar melhor caso a caso
- ◆ Como padronização deve-se iniciar o nome de métodos/atributos com minúscula. Se houver concatenações colocar as iniciais em maiúscula



Funcionario.h

```
class Funcionario {  
private:
```

```
    int codigo;  
    std::string nome;  
    int idade;
```

```
public:
```

```
    Funcionario(int c, std::string n, int i); // Construtor  
    void apresentar(); // Método para exibir informações
```

```
    void setCodigo(int c);  
    int getCodigo();  
    void setNome(std::string nome);  
    std::string getNome();  
    void setIdade(int idade);  
    int getIdade();
```

```
};
```



Funcionario.cpp

```
#include "Funcionario.h"
```

```
Funcionario::Funcionario(int c, std::string n, int i)
    : codigo(c), nome(n), idade(i) {}
```

```
void Funcionario::apresentar() {
    std::cout << "--- PRINTANDO DADOS ---" << std::endl;
    std::cout << "Codigo: " << codigo << std::endl;
    std::cout << "Nome: " << nome << std::endl;
    std::cout << "Idade: " << idade << std::endl;
```

```
}
```

```
void Funcionario::setCodigo(int codigo) {
    this->codigo = codigo;
```

```
}
```

```
int Funcionario::getCodigo() {
    return codigo;
```

```
}
```

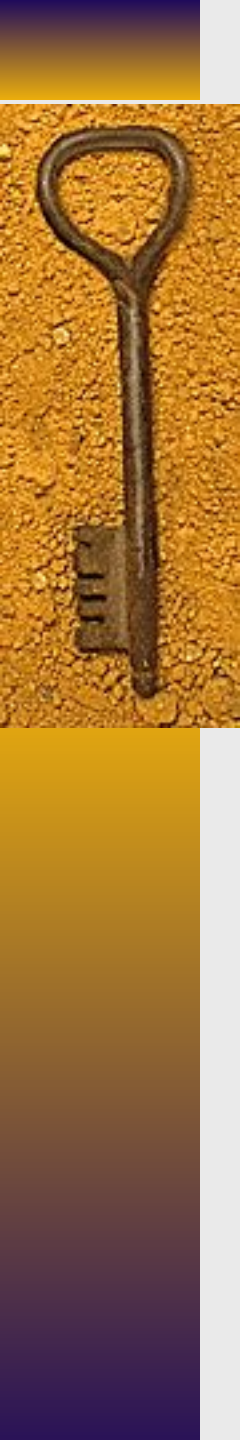
```
void Funcionario::setNome(std::string nome) {
    this->nome = nome;
```

```
}
```

```
std::string Funcionario::getNome() {
    return nome;
```

```
}
```

```
...
```



Como usar os sets() e gets() ?

```
#include "Funcionario.h"
```

```
int main() {
```

```
    Funcionario c (1, "Carlos", 30);  
    Funcionario r (2, "Roberta", 47);  
    Funcionario v (3, "Valter", 50);
```

```
    c.apresentar();  
    r.apresentar();  
    v.apresentar();
```

```
    std::cin.get(); // para a execução e espera uma entrada ....
```

```
    c.setIdade(32); //chamando o método set() para modificar a idade
```

```
    std::cout << "A idade do " << c.getNome() << " mudou para " <<  
    c.getIdade() << std::endl;
```

```
    return 0;
```





Operador de Referência → &

- ♦ Tem vários usos, vamos estudar um deles
- ♦ Veja a diferença
 - `int a = 10;`
 - `int& ref = a;`

- Criou-se uma variável chamada “a” com o valor de 10
- Criou-se uma referência chamada ref. Como ela não é uma variável, e sim uma referência, ela referencia (aponta) para a variável a.
- Se eu fizer `ref = 10`, eu estou na verdade atribuindo o valor 10 à variável a.



Operador de Referência → &

- ♦ Agora veja o uso em parâmetros de métodos
- ♦ Exemplo **sem** referência :

```
void alterar(int x) {  
    x = 100;  
    print (x); //imprime 100  
}
```

```
// chamando o alterar() pelo main()  
int a = 10;  
alterar(a);  
print (a) //imprime 10
```

- parâmetro x declarado normalmente (não é uma ref)
- cada variável continua com o seu próprio valor em seu escopo
- o x é outra variável, não tem ligação com a
- O que ocorre é $x = a$



Operador de Referência → &

- ♦ Agora veja o uso em parâmetros de métodos
- ♦ Exemplo **com** referência :

```
void alterar(int& x) {  
    x = 100;  
    print (x); //imprime 100  
}
```

```
// chamando o alterar() pelo main()  
int a = 10;  
alterar(a);  
print (a) //imprime 100
```

- parâmetro x declarado como ref para inteiro
- x agora aponta para a
- O que acontece é que não é mais uma atribuição simples de a para x, mas faz x apontar para o mesmo lugar de a
- Então quando se modifica x, está na verdade modificando quem o x se referencia



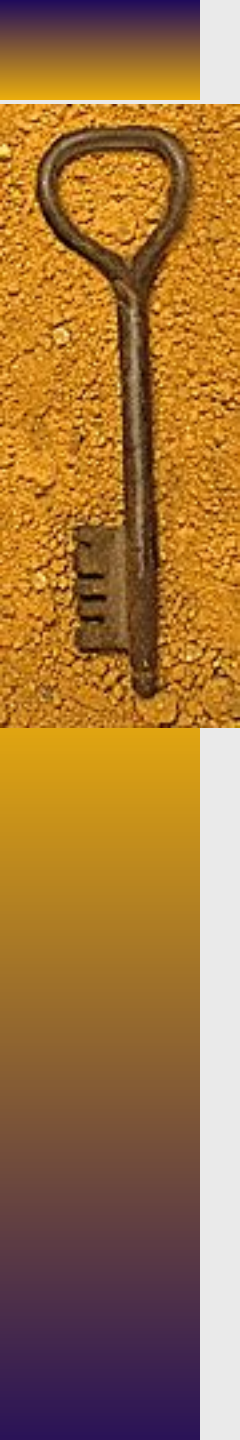
Operador de Referência → &

- ♦ Agora veja o uso em parâmetros de métodos
- ♦ Exemplo **com** referência :

```
void alterar(int& x) {  
    x = 100;  
    print (x); //imprime 100  
}
```

```
// chamando o alterar() pelo main()  
int a = 10;  
alterar(a);  
print (a) //imprime 100
```

Então cuidado ! o maior impacto é no código cliente, o código que usa o método. A variável que você passou pode ter o valor dela alterado depois de chamar o método.



Exercício 1

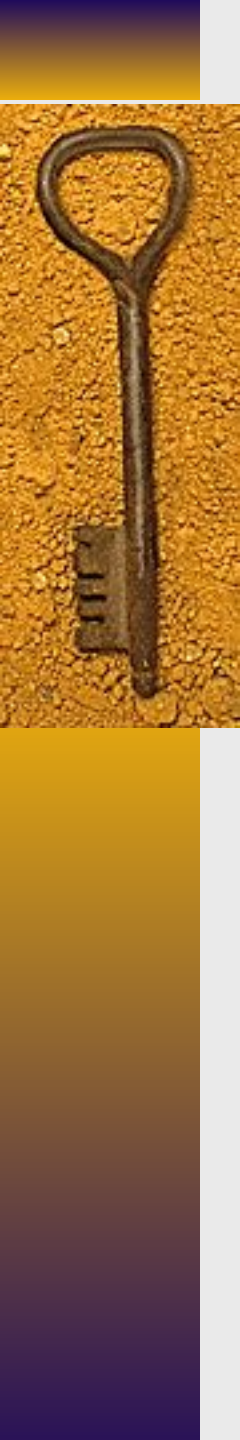
- ♦ Implemente uma classe chamada Produto
- ♦ Atributos:
 - código, nome, preço
- ♦ Métodos
 - construtor()
 - imprimir()
 - sets() e gets()
- ♦ Implemente um Main que faça o seguinte:
 - Instancie três produtos p1, p2 e p3 na stack
 - **Instancie dois produtos na heap p4 e p5**
 - use os métodos definidos na classe Produto (brinque com os métodos)
 - Tente acessar diretamente algum atributo privado



Produto

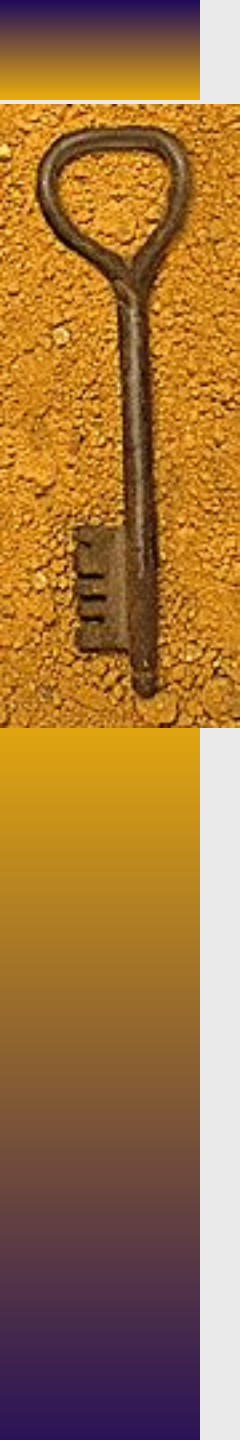
- codigo : int
- nome : std::string
- preco : int

- + <<construtor>> Produto(codigo : int, nome : string, preco : float) : Produto
- + <<construtor>> Produto(codigo : int, nome : string) : Produto
- + setCodigo(codigo : int) : void
- + setNome(nome : std::string) : void
- + setPreco(preco : float) : void
- + getCodigo() : int
- + getNome() : string
- + getPreco() : float



Exercício 1.1

- ♦ Faça um teste com passagem de parâmetro por valor e por referência. Para isso:
 - Crie um novo método na classe Produto chamado “testarPassagemPorReferência()”
 - Esse método deve receber como parâmetro um nome de produto (string) (**uma referência**)
 - Dentro do método, ele deve alterar esse parâmetro e imprimir o valor dele dentro do método.
 - Assim que ele der o rollback (volta o fluxo para o código cliente), você deve então verificar no código cliente (**Main**) o valor da variável que foi passada e ver se ela mudou. Se ela também mudou, é pq está correto.



Exercício 1.2

- ◆ Faça com que a classe Produto tenha agora um novo construtor
- ◆ Esse novo construtor deve aceitar apenas dois parâmetros: código e nome
- ◆ O valor do produto deve ser fixo, por exemplo, um valor de 100
- ◆ Para que serve isso?
 - Se você criar um produto passando só dois parâmetros, esse produto deve ser criado com um valor default
- ◆ Crie um novo produto no Main usando esse novo construtor e imprima os dados dele para ver se o valor ficou 100


```
Codigo: 1  
Nome: Monitor  
Preco: 900  
-----
```

```
Codigo: 2  
Nome: Teclado  
Preco: 150  
-----
```

```
Codigo: 3  
Nome: Mouse  
Preco: 80  
-----
```

```
Codigo: 4  
Nome: Notebook  
Preco: 3500  
-----
```

```
Codigo: 5  
Nome: Impressora  
Preco: 600  
-----
```

```
Codigo: 6  
Nome: monitor  
Preco: 100  
-----
```

```
Valor da variavel criada aqui no main:  valor definido no main
```

```
O valor aqui dentro é: mudei !
```

```
Valor da variavel criada aqui no main (depois da chamada):  mudei !
```

Ao executar o programa,
esta deve ser uma possível
saída

Exercício 2 - Operador &

Implemente um programa em C++ que:

1. Declare duas variáveis inteiras no main e atribua valores iniciais a elas.
2. Implemente duas funções:
 - **Função 1:**
Deve receber um número inteiro por **valor** e somar 10 a ele.
 - **Função 2:**
Deve receber um número inteiro por **referência** e somar 10 a ele.
3. No método main:
 - Chame as duas funções, passando as variáveis criadas.
 - Exiba os valores das variáveis **antes e depois** das chamadas das funções.



